Amortized Custom eFPGA Layout Search via Reinforcement Learning: Zero-Shot Generalization Across Circuit Benchmarks

Anonymous Author(s)

Abstract

Embedding a small reconfigurable fabric into an ASIC to implement one security-critical IP block is a known defense against reverse engineering, since the protected function is absent from the fixed silicon until configured. Because this use case targets one known netlist rather than general-purpose reconfigurability, a custom-tailored fabric layout can recover much of the area/delay/power overhead a generic FPGA would otherwise impose. Prior work (GOLDS and a follow-up NSGA-II method) searches for such layouts with an evolutionary algorithm re-initialized from scratch for every new netlist. We instead train a reinforcement-learning policy, conditioned on a graph-neural-network encoding of the target netlist, to choose DSP/BRAM placement and fabric aspect ratio, evaluated directly against the VTR synthesis-to-routing flow. Under a controlled comparison against our own genetic algorithm with an identical oracle and metric, the policy reaches 74.06% ADP reduction against the GA's 68.17%. Jointly trained across 11 circuits, it reaches a 27.35% aggregate in-pool ADP reduction averaged over three training seeds. Recomputed on GOLDS' published metric and baseline, a zero-shot evaluation with no training on the target benchmark reaches 37.4% against GOLDS' reported ~35%. Across the three seeds we further characterize result reliability and find a marked asymmetry: in-pool performance is substantially more seed-sensitive than zero-shot generalization, with two in-pool benchmarks spanning ~28 points across seeds while every held-out benchmark spans at most ~12. We report the individual per-seed results rather than a mean ± SD that three runs cannot justify.

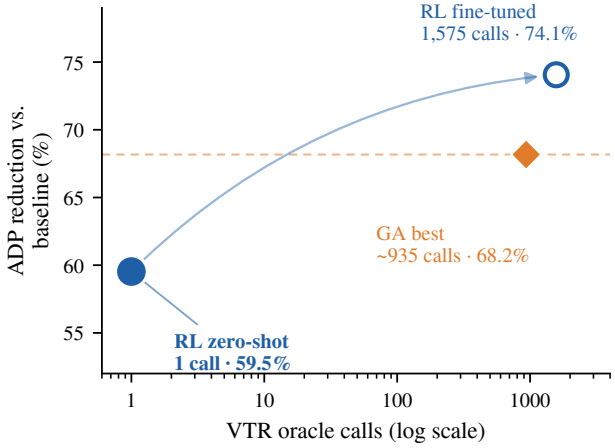

Figure 1: Amortization on a previously unseen netlist (`custom_macbuf`). A single zero-shot rollout of the pretrained policy reaches 59.5% ADP reduction at one VTR oracle call (lower left); fine-tuning continues from that same policy to 74.1% at 1,575 calls, exceeding the 68.2% best of our genetic algorithm (dashed line) reached only after ~935 calls. The horizontal axis is oracle calls on a log scale; all three points are measured against the same traditional baseline (ADP = 44,503).

1 Introduction

Embedding a small, reconfigurable fabric into an ASIC to implement one security-critical IP block — so that the function is not recoverable from the silicon layout or GDSII without the bitstream — is a known defense against reverse engineering and IP theft [20]. A generic, general-purpose FPGA architecture is built to serve arbitrary circuits, and is therefore over-provisioned — extra LUTs, routing, and DSP/BRAM capacity — for any one fixed design that will only ever run a single bitstream. Because the eFPGA masking use case targets exactly one known application on a small embedded fabric, rather than general-purpose reconfigurability, a custom-tailored architecture and placement, sized and arranged specifically for that one netlist, is viable, and can recover much of the area/delay/power overhead a generic FPGA would otherwise impose. We use the product of area, delay, and power (ADP) as our optimization target throughout this paper because it is precisely the size of this overhead — the "masking tax" of moving a protected block onto reconfigurable fabric instead of hardening it directly into standard cells.

Two prior papers establish that per-instance search over DSP/BRAM placement (and, in one case, CLB/arithmetic-block sizing) can substantially reduce this tax: GOLDS [20] uses a genetic algorithm with an Area×Delay fitness function and reports up to ~35% ADP improvement on the `diffeq1` benchmark; a follow-up applies NSGA-II to a larger, multi-objective search space that additionally co-evolves CLB LUT size and custom arithmetic-block sizing [3]. Both, however, are *testbench-specific*: the genetic algorithm's population is re-initialized and re-evolved from scratch for every new protected IP block, and every generation of every run requires its own full, expensive VTR evaluation. Nothing learned while optimizing one netlist's layout carries over to the next netlist — each new target benchmark pays the full search cost again, with no way to amortize the work already done on previous designs.

This paper asks whether a *learned policy* can replace that per-instance search loop. RL placing blocks onto an *already-fixed* FPGA fabric is not new; we pose two questions instead. **RQ1**: can the same RL machinery *construct* the custom fabric layout itself — DSP/BRAM placement and aspect ratio for an embedded FPGA built around one netlist — the regime where a per-instance GA has so far been the only option? **RQ2**: can such a policy be *benchmark-agnostic*, pretrained across a pool of netlists so the same checkpoint speeds up layout for a previously unseen benchmark (zero-shot or after brief fine-tuning) rather than re-searching from scratch?

We train a single reinforcement-learning policy, conditioned on a graph-neural-network encoding of the target netlist, to place

DSP/BRAM hard blocks and choose a fabric aspect ratio, evaluated directly against the real VTR (Yosys + VPR) synthesis-to-routing flow rather than a learned or analytic proxy. Because the same checkpoint applies to a netlist it has never seen without re-running search, the cost of finding a good layout is amortized across deployments. The objective is therefore not to win every single-instance ADP comparison outright, but to produce a strong layout for a previously unseen netlist at near-zero search cost (Figure 1).

The paper sits at the intersection of two lines of work not previously connected (Section 2): custom eFPGA layout search for logic masking, which to date has used only per-instance evolutionary search [3, 20], and learned policies for physical design, which have targeted ASIC macro placement at a much larger scale and through a learned reward proxy rather than the true toolchain [16, 18]. We bring the amortization argument from the latter to the former problem, evaluated end-to-end against the same VTR oracle the former already established as the fidelity bar.

Concretely, this paper contributes:

- A reinforcement-learning formulation of custom eFPGA DSP/BRAM placement and aspect-ratio selection, with a GCN-based netlist encoding (Section 3.3) and a fixed-size policy that remains loadable against benchmarks never seen during training (Section 3.4), evaluated directly against the same VTR-based oracle used by prior eFPGA-layout search work.
- A controlled comparison against a from-scratch genetic algorithm under an identical evaluation oracle, ADP metric, and baseline, on a held-out benchmark: 74.06% ADP reduction for RL against 68.17% for the GA (Section 5.2).
- A joint 11-benchmark training result (27.35% aggregate in-pool ADP reduction, averaged over three seeds) (Section 5.1), and a literature-facing comparison against GOLDS' published metric and baseline, recomputed on a zero-shot evaluation: 37.4% against GOLDS' ∼35% (Section 5.3).
- A three-seed reliability analysis exposing an asymmetry: in-pool fitting is far more seed-sensitive (two benchmarks spanning ∼28 points across seeds) than zero-shot generalization (at most ∼12 points across all six held-out benchmarks), reported as raw per-seed results rather than mean ± SD (Sections 5.4, 5.6).

2 Background and Related Work

2.1 eFPGA-Based Logic Masking

Embedding a reconfigurable fabric into an ASIC to hide one IP block's function behind a bitstream is a hardware-security technique distinct from standard logic locking or camouflaging, since the protected function is not merely obfuscated in the netlist but physically absent from the fixed silicon until configured. Automated flows for partitioning RTL into embedded reconfigurable fabrics established eFPGA redaction as a practical defense [6, 21], and subsequent work has cut its overhead by shrinking under-utilized routing [15] or de-regularizing the fabric itself [8] — motivating the custom, non-island-style layouts this paper synthesizes. These flows all rely on a non-learning search (heuristic or evolutionary) re-run per design; none amortizes layout effort across netlists. GOLDS [20] introduced custom-tailored eFPGA layout design for this purpose: a genetic algorithm searches over DSP/BRAM tile placement on a fixed-size fabric, with an Area×Delay fitness function evaluated through the real VTR flow, and reports up to ∼35% ADP-relevant improvement over a traditional island-style layout on `diffeq1`. A follow-up [3] extends this to a multi-objective NSGA-II search that additionally co-evolves CLB LUT size and custom DSP/BRAM block sizing — not just their placement — optimizing delay and power jointly and reporting up to ∼44.6% improvement on a larger design-space. Both papers' searches are run fresh, from a random population, for every new protected design; neither carries information learned on one IP block forward to the next. This paper targets the same problem GOLDS defines (custom DSP/BRAM placement plus aspect ratio, evaluated through the same VTR-based oracle) and treats it as the narrower, more directly comparable target for our controlled experiments (Section 5.2), while citing the NSGA-II extension as context for the larger design space a future version of this approach could also cover (Section 6).

2.2 Learned Policies for Physical Design

Reinforcement learning has been applied to chip-scale physical design. AlphaChip [11, 18] trains an RL agent to place ASIC macro blocks using a learned reward proxy, and reports policies that transfer across chip designs without retraining — the amortization argument we make here, at a much larger design scale. Subsequent work explores visual state representations [17], joint placement-and-routing [5], and offline RL over a transformer policy [16]. All motivate the claim this paper tests in a smaller-scale setting: a trained policy can out-amortize per-instance search once many designs must be placed. Unlike AlphaChip, we evaluate every candidate against the true VTR flow rather than a learned proxy, trading speed for fidelity — affordable at our scale, where a VTR run completes in seconds to low minutes (Section 4.1).

Within the FPGA setting specifically, RL has guided placement onto an *already-fabricated* grid — selecting annealing perturbations in VPR [10] or driving a Gym-style VPR environment [4] — but always optimizes the assignment of blocks to a fixed grid, never its geometry. Transferable EDA policies have likewise appeared for tool tuning [13] and global placement [12], but not, to our knowledge, for *synthesizing* a custom FPGA fabric itself, which is what our policy does.

2.3 VTR/VPR Background

We build on the open-source Verilog-to-Routing (VTR) [2, 9, 19] toolchain: Yosys performs synthesis (via the VTR integration [7]) and VPR performs packing, placement, and routing against an architecture described in an XML schema of tile types, positions, and connectivity. This architecture XML is the artifact GOLDS and the NSGA-II follow-up generate by hand-coded search and that we generate by policy rollout (Section 4.1); VTR's acceptance of an arbitrary heterogeneous tile layout, not only the regular island-style grids it ships, is what makes this entire line of work possible.

3 Methodology

3.1 Problem Formulation

Given a fixed eFPGA fabric template parameterized by a tile grid of width W and height H, and a netlist requiring n_{dsp} DSP and

n_{bram} BRAM hard blocks, our goal is to choose a placement $\pi = (\text{ar}, \{(x_i, y_i)\}_{i=1}^{n_{\text{dsp}}+n_{\text{bram}}})$ — an aspect ratio $\text{ar} \in \mathcal{R}$ for the fabric plus a tile coordinate for every hard block — that minimizes the post-synthesis, post-place-and-route cost of implementing that netlist on the resulting custom architecture. We use the product of routing area, critical-path delay, and power (ADP) as the cost, since this directly reflects the silicon and timing overhead of moving the protected IP block onto reconfigurable fabric:

$$\text{ADP}(\pi) = \text{Area}(\pi) \times \text{Delay}(\pi) \times \text{Power}(\pi), \tag{1}$$

where each term is obtained by running the candidate placement through the full VTR (Yosys + VPR) synthesis-to-routing flow (Section 4). We report results as percentage ADP reduction relative to a traditional, non-customized island-style eFPGA baseline at the same logic capacity.

3.2 RL Formulation

We pose placement as a finite-horizon sequential decision process and train a policy with Proximal Policy Optimization (PPO), using the maskable variant [14] (`sb3_contrib`'s `MaskablePPO`) so that the action space can be restricted to currently-legal tile coordinates at every step.

Episode structure. An episode places exactly one benchmark's blocks onto the shared canvas. Step 0 selects the fabric aspect ratio from a fixed discrete set $\mathcal{R}$; subsequent steps place each DSP, then each BRAM instance, one per step, in descending order of shared-net connectivity. Once every block is placed, the environment bakes the resulting layout into an architecture XML and constraints file (via Jinja2 templating) and invokes the VTR flow to obtain $\text{Area}(\pi)$, $\text{Delay}(\pi)$, and $\text{Power}(\pi)$; all intermediate steps receive zero reward.

Action space. The action space is $\text{Discrete}(W_{\max} \times H_{\max})$, fixed across all benchmarks in a training run (Section 3.4). At step 0 an action index selects an entry in $\mathcal{R}$; at later steps an action index decodes to a tile coordinate (x, y) via a fixed stride on $H_{\max}$, independent of which benchmark is active. Action masking — keyed off the *active* benchmark's real width/height and the blocks already placed — restricts the policy to legal, non-overlapping, in-bounds coordinates for that specific benchmark and step.

Reward. The terminal reward is a weighted sum of negative log-ratios against the traditional baseline for the active benchmark:

$$r = -\left(w_a \log \frac{\text{Area}}{\text{Area}_0} + w_d \log \frac{\text{Delay}}{\text{Delay}_0} + w_p \log \frac{\text{Power}}{\text{Power}_0}\right), \tag{2}$$

where $\text{Area}_0, \text{Delay}_0, \text{Power}_0$ are the traditional-baseline metrics for that benchmark and w_a, w_d, w_p are configurable weights (Section 3.5). With $w_a = w_d = w_p = 1$, r equals $\log(\text{ADP}_0/\text{ADP})$ — i.e. the reward *is* the log of the ADP-improvement ratio in Eq. 1, making reward and our headline metric directly comparable. An invalid choice (degenerate aspect ratio, out-of-bounds or overlapping placement, or a VTR flow failure) terminates the episode early with a fixed penalty reward of -10.

3.3 GCN Feature Extractor

To let a single policy generalize across benchmarks with different netlist sizes, hard-block counts, and grid dimensions, we encode each netlist as a graph rather than a flat vector, giving the policy the inductive bias to transfer across circuit topologies that graph learning provides elsewhere in EDA [22]. We first reduce each benchmark's full netlist graph to a compact, fixed-schema representation: one node per DSP/BRAM hard-block instance, plus one aggregate "fabric" node summarizing the remaining standard-cell logic. Each node carries a 4-dimensional feature vector `[is_dsp, is_bram, is_fabric, net_count_normalized]`. Edges between two hard-block nodes are weighted by their raw shared-net count; edges between a hard-block node and the fabric node are weighted by the summed shared-net count across the standard cells that block touches. As these two edge kinds live on very different scales, we normalize each into $[0, 1]$ separately by a log compression $w_{\text{norm}} = \log(1 + w)/\log(1 + c)$ with a per-kind ceiling c from the training pool, so neither type saturates or vanishes.

This reduced graph is padded to a fixed (MAX_NODES, MAX_EDGES) — computed once over the full benchmark *universe* (Section 3.4) — and fed to a small GNN feature extractor shared by the policy and value heads: two `GCNConv` layers (hidden width 64) produce per-node embeddings h_i; a global mean pool gives a whole-netlist embedding, and the embedding of the node being placed this step gives a focused per-step embedding. These are concatenated with a small CNN over the occupancy grid (4 channels: occupancy, block-type hint, aspect ratio, net count) and the active benchmark's normalized (width, height), then projected to a 128-dimensional vector for the PPO policy and value heads. The extractor is trained end-to-end under the PPO loss, with no separate GNN pretraining.

3.4 Universe Superset and Action Masking

A policy's input/output dimensions are fixed at construction, yet we want one checkpoint loadable against benchmarks never trained on, including ones discovered later. We therefore size every dimension-dependent quantity — canvas ($W_{\max}, H_{\max}$), MAX_NODES, MAX_EDGES — from a benchmark *universe* that is a deliberate superset of the sampled training set, including benchmarks reserved purely for zero-shot evaluation, provided none exceeds the shared dimensions. Per-episode action masking (Section 3.2) and the `valid_wh` observation let the same fixed-size network operate on each benchmark's smaller real footprint within that canvas.

3.5 Training Setup

Each training run samples uniformly across the 11-benchmark training pool (Table 1) episode-by-episode, so a single policy is updated jointly on all 11 circuits rather than one at a time. We use `MaskablePPO` with 24 parallel environments, 48 rollout steps per environment per update, batch size 144, 15 epochs per update, entropy coefficient 0.01, and reward weights $w_a = w_d = w_p = 1$ (Eq. 2), giving reward = $\log(\text{ADP}_0/\text{ADP})$. Each run trains for 13,000 episodes (capped independently of total timesteps) with a per-episode VTR timeout of 300s to keep a degenerate early-exploration episode from stalling the other 23 parallel workers. We report results from three independent training seeds (7, 42, 123) under this identical configuration, used throughout Section 5 to characterize result reliability rather than report a single run's numbers as ground truth.

4 Experimental Setup

4.1 Evaluation Oracle

Every candidate placement is scored by the true downstream toolchain, not a proxy. A placement (aspect ratio + hard-block coordinates) is rendered into a complete VTR architecture XML and a VPR placement-constraints file via Jinja2 templates parameterized by tile coordinates, block type, and fabric aspect ratio. This architecture and the benchmark's Verilog are then run through the full VTR flow — Yosys-based synthesis, VPR packing, placement, and routing, followed by VPR's power estimator against a 45nm PTM technology file — and we extract routing area, critical-path delay, and dynamic power from the resulting VPR reports. All evaluations use a VTR 9.0.0 development build (git `22a09d39`, GCC 13.3.0). Evaluated (placement, benchmark) pairs are cached, since the synthesis-to-routing flow is the dominant cost of both training and evaluation (seconds to low minutes per call depending on benchmark size).

4.2 Baseline

Our traditional baseline is VTR's default island-style architecture template (`k6_frac_N10_mem32K_40nm`: 6-input LUTs, cluster size 10, 32K-bit BRAMs, 40nm) at matched logic capacity for each benchmark — the same template used by GOLDS [20] as its traditional-layout comparison point, which lets us compare percentage improvements directly against theirs on a shared starting point (Section 5.3).

4.3 Benchmarks

Table 1 lists the benchmark pool used in this work: an 11-benchmark training set of real circuits spanning a range of DSP/BRAM resource profiles, and a held-out set used only for zero-shot evaluation (Section 5.4), never sampled during training. Training circuits come from the standard VTR suite [19]; two held-out benchmarks (`softmax`, `reduction_layer`) are deep-learning workloads from Koios [1], and the remaining four are hand-written probes — including `custom_macbuf`, an internally built multiply-accumulate buffer used in the GA comparison (Section 5.2) — stressing DSP/BRAM mixes rare in the real circuits.

4.4 Baselines Compared

We compare against: (1) the traditional island-style eFPGA baseline (Section 4.2); (2) a genetic algorithm we implemented under the same evaluation oracle and ADP metric (Section 5.2); and (3) published results from GOLDS [20] and the NSGA-II follow-up [3], subject to the metric caveat in Section 5.3.

5 Results

5.1 In-Pool Performance

Figure 2 (upper group) reports per-benchmark in-pool ADP reduction for the 11-benchmark policy on its own training set across three independent seeds (7, 42, 123; Section 3.5). We show all three per-seed outcomes rather than a mean ± SD, which three runs cannot justify (Section 5.6). Each seed's pooled aggregate — one minus the ratio of summed ADP across all 11 benchmarks to summed baseline ADP, *not* a simple mean of per-benchmark percentages — is 35.73%, 23.11%, and 23.97% (mean 27.35%). `raygentop` is a persistent laggard, every seed within $[-2.2, -1.7]$% and never positive. Its 6-DSP/1-BRAM profile is the only one in the pool mixing a large DSP count with a BRAM (other DSP-heavy benchmarks have no BRAM; other BRAM-having ones have ≤ 1 DSP), and despite a fair $\approx$9.1% episode share the policy never solves this joint placement — plausibly a representational gap rather than a convergence shortfall, and our most immediate diagnostic target. Two benchmarks, `or1200` and `boundtop`, account for almost all of the seed-to-seed instability: their per-seed outcomes span 5.8–33.5% and 21.3–50.7% respectively, ranges three to four times wider than any other benchmark in the pool (all ≤ 8 points). We analyze this asymmetry in Section 5.6.

Table 1: Benchmark pool: grid size and hard-block requirements.

Benchmark	Grid	DSP	BRAM
Training set (11 benchmarks)			
fifo	6×6	0	1
ch_intrinsics	10×10	0	1
diffeq2	14×14	5	0
boundtop	13×13	0	1
diffeq1	14×14	5	0
spree	12×12	1	3
mkSMAdapter4B	18×18	0	5
or1200	25×25	1	2
mmc_core	13×13	0	1
mkPktMerge	26×26	0	15
raygentop	17×17	6	1
Held-out (zero-shot only)			
softmax	41×41	8	0
reduction_layer	42×42	0	32
lightweight_cipher	12×12	0	3
custom_macbuf	12×12	3	1
mkDelayWorker32B	48×48	0	43
arm_core	35×35	0	24

5.2 GA Head-to-Head Comparisons

For a controlled comparison against evolutionary search — without the metric- and baseline-matching caveat that attaches to the GOLDS comparison (Section 5.3) — we implemented a genetic algorithm under the same evaluation oracle, ADP metric, and baseline as the RL pipeline, and ran it on `custom_macbuf` (held out of RL training; Table 1): roulette-wheel selection, single-point crossover, per-gene mutation over both hard-block coordinates and fabric aspect ratio. We use a population of 5 to match the per-oracle-call budget available during an RL rollout (one VTR call per episode step, one candidate placement per individual per generation); the cap is 500 generations with early-stop patience of 50 generations without improvement (converged at generation 186, consuming approximately 935 VTR calls). The RL comparison point is the 11-benchmark policy (Section 5.4) fine-tuned on `custom_macbuf` alone. Table 2 reports the result: RL reaches **74.06%** ADP reduction versus the GA's **68.17%** on the identical benchmark, metric, and toolchain.

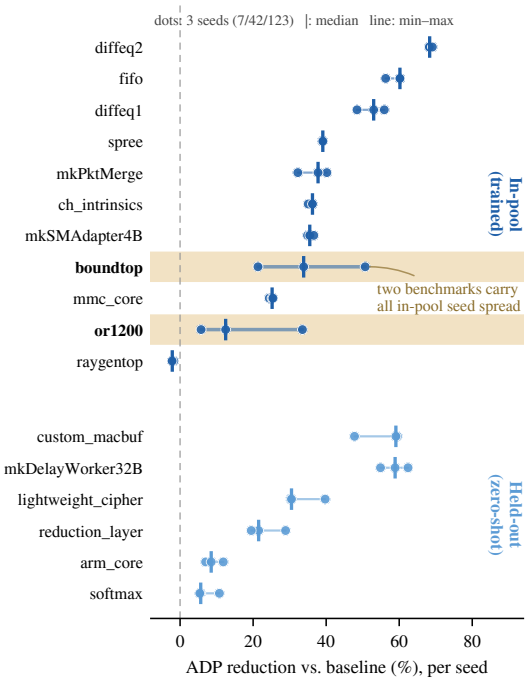

Figure 2: Per-seed ADP reduction for every benchmark; this figure carries the per-benchmark results in place of a table. Each benchmark shows its three training-seed outcomes (dots; seeds 7/42/123), the median (tick), and the min–max range (line) — raw seeds rather than mean ± SD, which three runs cannot justify. In-pool fitting is seed-sensitive on two benchmarks, `or1200` and `boundtop` (shaded), whose ranges span ∼28 and ∼29 points; every other in-pool benchmark and all six held-out (zero-shot) benchmarks are tight (range ≤ 12 points). The held-out group is more seed-stable than in-pool fitting.

Table 2: GA vs. RL on `custom_macbuf` (baseline ADP = 44,503). GA: population 5, ≤500 generations, early-stop patience 50. RL zero-shot is a single deterministic rollout (seed 42); RL fine-tuned continues the 11-benchmark policy on `custom_macbuf`.

Method	Best ADP	Reduction	VTR calls
GA	14,164.5	68.17%	∼935
RL zero-shot	18,008.4	59.53%	**1**
RL fine-tuned	11,553.2	**74.06%**	1,575

Oracle cost. The GA uses approximately 935 VTR oracle calls (population 5 over 186 generations) to reach its best `custom_macbuf` layout. Applied zero-shot, the policy reaches 59.53% reduction in a single oracle call; fine-tuning then uses 1,575 unique calls (of 6,000 episodes, the remaining 74.6% resolved as cache hits as the policy's entropy falls) to reach 74.06%, exceeding the GA at 40% fewer oracle calls. Figure 3 shows the resulting layout: the policy compacts the baseline's 10×12 active region into 6×7, which accounts for most of the ADP reduction.

5.3 Head-to-Head Against GOLDS

As a secondary, literature-facing comparison, we recompute a genuine zero-shot result — a policy trained only on `diffeq2`, evaluated zero-shot on `diffeq1` with no training on it whatsoever — on GOLDS' [20] own 2-term Area×Delay metric (rather than our 3-term Area×Delay×Power metric used elsewhere), under the same traditional baseline architecture GOLDS uses. This yields a **37.4%** reduction versus GOLDS' published best-of-population result of ∼**35%** on the same benchmark — a real, if narrower, margin, achieved with zero benchmark-specific training. We report this result separately from Section 5.2 precisely because it required recomputing on a different metric than the rest of this paper; nowhere else do we compare 2-term GA numbers against our 3-term RL numbers directly.

5.4 Zero-Shot Generalization

Figure 2 (lower group) reports ADP reduction on all six held-out benchmarks (Table 1), across the same three seeds as Section 5.1, evaluated with zero benchmark-specific training. Two of the six (`softmax`, `reduction_layer`) were part of the training-time *universe* used to size the policy's tensors (Section 3.4) but never sampled during training; the other four were entirely outside the universe as well, a strictly harder generalization test. Every benchmark finishes positive under every seed — no sign flips at all, unlike the in-pool result — and every per-seed range is narrow (at most $\approx$ 12 points, for `custom_macbuf`), with no benchmark approaching the `or1200`/`boundtop` spread. The overall average across all six benchmarks and three seeds is **31.23%**. The strongest, most reliable benchmarks (`mkDelayWorker32B`, `custom_macbuf`) are also the most seed-stable in relative terms (coefficient of variation 5–10%), while the weakest (`softmax`, `arm_core`) are proportionally the least certain (coefficient of variation 22–34%) — the absolute spread is small everywhere, but on a small mean that spread represents a much larger fraction of the result.

A note on deterministic decoding. Every number above uses greedy decoding (the policy's most-likely action at each step), which is the intended deployment mode: a layout is generated once and baked into the architecture, not sampled. As a diagnostic, we drew 10 *stochastic* rollouts per benchmark on one seed; their average ADP reduction was negative for 4 of 6 benchmarks despite every deterministic result being positive. This gap is expected and does not invalidate the deterministic numbers — it shows the learned distribution is concentrated, with most mass away from the high-quality region the top choice occupies. The zero-shot results should thus be read as the top choice's performance under each seed, not an expectation over sampling; a full three-seed stochastic evaluation is left to future work.

5.5 Fine-Tuning Efficiency

We also characterize continued learning on a partially-known benchmark (`diffeq1`, in the training pool): fine-tuning the multi-benchmark policy versus training from scratch, both 6,000 episodes under identical hyperparameters and separate caches. The fine-tuned

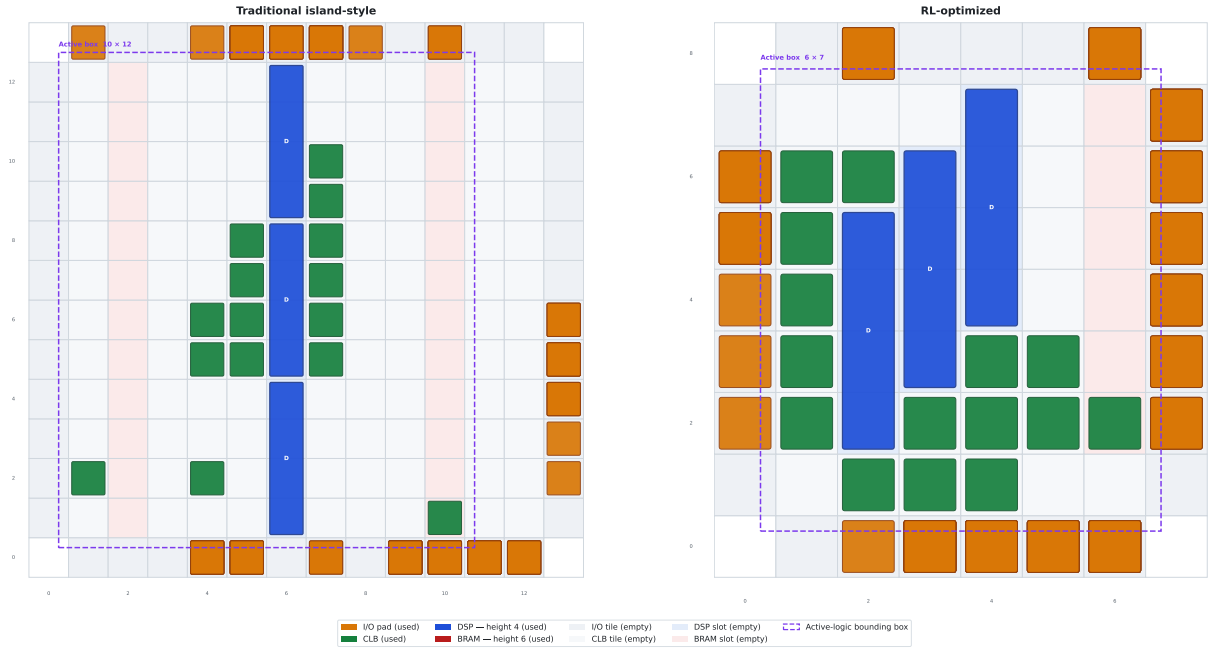

Figure 3: What the policy recovers, physically (`custom_macbuf`, the GA comparison benchmark). The traditional island-style baseline (left) spreads the same netlist's logic across a 10×12 active region; the RL-optimized fabric (right) packs it into a 6×7 region by clustering the DSP columns with their surrounding CLBs. Tile types: DSP (blue, four tiles tall), CLB (green), I/O pad (orange); the dashed rectangle marks the active-logic bounding box. This compaction of the routed area is the dominant term in the ADP reductions reported in Figure 2.

run dominates typical reward throughout (0.785 vs. 0.736 in the final window) and plateaus by ≈episode 3,000, while scratch never plateaus within its budget. The mechanism is entropy collapse: the fine-tuned policy's entropy falls from 2.00 to 0.79 (vs. scratch's 4.63 to 2.42), so only 31.5% of its episodes are novel VTR evaluations versus 94.4% for scratch — a sample-efficiency win that, as Section 5.6 notes, also caps further exploration beyond the pretrained prior. This is continued learning, not cross-benchmark transfer (Section 5.4).

5.6 Seed Variance: In-Pool vs. Zero-Shot

Across three independent training seeds, the 11-benchmark policy exhibits a clear asymmetry: *in-pool* performance is considerably more seed-sensitive than *zero-shot* performance. For nine of the eleven training benchmarks, the per-seed range is small (≤ 8 points); two benchmarks, `or1200` and `boundtop`, span ~28 and ~29 points respectively — three to four times the spread of any other benchmark — and account for almost the entire spread in the pooled aggregate (35.73% / 23.11% / 23.97% across seeds). On the six held-out zero-shot benchmarks, by contrast, every per-seed range stays within ~12 points (Figure 2); the two shaded in-pool benchmarks have the widest ranges in either group.

We do not have a confirmed mechanism but propose one. In-pool performance depends on which benchmarks a given seed converges on slowly within the shared episode budget, whereas zero-shot performance reflects a more general property of the learned representation (Section 3.3), independent of any one benchmark's convergence. The entropy collapse of Section 5.5 compounds this: a seed converging early on most benchmarks retains fewer episodes for a slow one like `or1200`. No seed leads on both volatile benchmarks and elsewhere at once (seed 42 leads on `or1200` and `boundtop`; seeds 7 and 123 lead on others), pointing to per-benchmark convergence variance rather than a globally better seed. At $n = 3$ this is a characterization; more seeds would establish whether the asymmetry is stable.

6 Conclusion

We asked whether a learned policy can replace per-instance evolutionary search for custom eFPGA-based logic masking. Under a controlled comparison on an identical benchmark, oracle, ADP metric, and baseline, the RL policy exceeds our genetic algorithm (74.06% against 68.17% ADP reduction); a single zero-shot rollout already provides a useful layout at one oracle call against the GA's ~935, and fine-tuning reaches a better optimum at lower oracle cost (Section 5.2). The same policy generalizes zero-shot to six held-out benchmarks and, recomputed on GOLDS' published metric, exceeds GOLDS' reported result (Sections 5.3, 5.4).

Two reliability findings qualify these results and matter for reproduction. In-pool fitting is far more seed-sensitive than zero-shot transfer (the aggregate ranges 23.11–35.73% across seeds, driven by `or1200` and `boundtop`), and the deterministic zero-shot numbers diverge from stochastic sampling (negative for 4 of 6 benchmarks in a preliminary check). A single-seed or single-rollout number is therefore not a stable characterization of the policy (Section 5.6).

Several gaps remain. Results rest on three seeds rather than the larger seed-averaged statistics a fully rigorous claim requires, and a systematic stochastic-rollout evaluation across all seeds is still outstanding. The action space covers DSP/BRAM placement and aspect ratio only; the NSGA-II follow-up [3] additionally co-evolves CLB LUT size and arithmetic-block sizing, a natural extension given that the GCN representation (Section 3.3) already generalizes across netlist size and resource mix. Finally, the benchmark pool does not yet cover the full set GOLDS and the NSGA-II follow-up report on (e.g. sha, the Koios suite); broadening it would let every comparison here be made directly rather than through the metric-converted comparison of Section 5.3.

References

[1] Aman Arora, Andrew Boutros, Daniel Rauch, Aishwarya Rajen, Aatman Borda, Seyed A. Damghani, Samidh Mehta, Sangram Kate, Pragnesh Patel, Kenneth B. Kent, Vaughn Betz, and Lizy K. John. 2021. Koios: A Deep Learning Benchmark Suite for FPGA Architecture and CAD Research. In *31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE. doi:10.1109/FPL53798.2021.00010

[2] Vaughn Betz and Jonathan Rose. 1997. VPR: A New Packing, Placement and Routing Tool for FPGA Research. In *7th International Workshop on Field-Programmable Logic and Applications (FPL)*. Springer-Verlag, 213–222. doi:10.1007/3-540-63465-7_226

[3] D. V. Bhargav, G. Pradyumna, and Madhav Rao. 2025. Meta-Heuristic Optimization of Custom Heterogeneous Blocks Defined eFPGA Design. In *38th International Conference on VLSI Design (VLSID)*. IEEE, 326–331. doi:10.1109/VLSID64188.2025.00069

[4] Ruichen Chen, Shengyao Lu, Mohamed A. Elgammal, Peter Chun, Vaughn Betz, and Di Niu. 2023. VPR-Gym: A Platform for Exploring AI Techniques in FPGA Placement Optimization. In *33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 72–78. doi:10.1109/FPL60245.2023.00018

[5] Ruoyu Cheng and Junchi Yan. 2021. On Joint Learning for Solving Placement and Routing in Chip Design. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 16508–16519.

[6] Luca Collini, Jitendra Bhandari, Chiara Muscari Tomajoli, Abdul Khader Thalakkattu Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2025. ARIANNA: An Automatic Design Flow for Fabric Customization and eFPGA Redaction. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* (2025). volume/issue/pages unverified at press time. doi:10.1145/3737287

[7] Seyed Alireza Damghani and Kenneth B. Kent. 2022. Yosys+Odin-II: The Odin-II Partial Mapper with Yosys Coarse-grained Netlists in VTR. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. 157. doi:10.1145/3490422.3502344

[8] Voktho Das, Kimia Zamiri Azar, and Hadi M. Kamali. 2026. NuRedact: Non-Uniform eFPGA Architecture for Low-Overhead and Secure IP Redaction. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE.

[9] Mohamed A. Elgammal, Amin Mohaghegh, Soheil G. Shahrouz, Fatemehsadat Mahmoudi, Fahrican Kosar, Kimia Talaei, Joshua Fife, Daniel Khadivi, Kevin E. Murray, Andrew Boutros, Kenneth B. Kent, Jeffrey B. Goeders, and Vaughn Betz. 2025. VTR 9: Open-Source CAD for Fabric and Beyond FPGA Architecture Exploration. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 18, 3 (2025), 39:1–39:53. doi:10.1145/3734798

[10] Mohamed A. Elgammal, Kevin E. Murray, and Vaughn Betz. 2022. RLPlace: Using Reinforcement Learning and Smart Perturbations to Optimize FPGA Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 41, 8 (2022), 2532–2545. doi:10.1109/TCAD.2021.3109863

[11] Anna Goldie, Azalia Mirhoseini, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2024. Addendum: A graph placement methodology for fast chip design. *Nature* 634, 8034 (2024), E10–E11. doi:10.1038/s41586-024-08032-5

[12] Yunbo Hou, Haoran Ye, Shuwen Yang, Yingxue Zhang, Siyuan Xu, and Guojie Song. 2025. TransPlace: Transferable Circuit Global Placement via Graph Neural Network. *arXiv preprint arXiv:2501.05667* (2025).

[13] Hao-Hsiang Hsiao, Yi-Chen Lu, Pruek Vanna-Iampikul, and Sung Kyu Lim. 2024. FastTuner: Transferable Physical Design Parameter Optimization using Fast Reinforcement Learning. In *International Symposium on Physical Design (ISPD)*. ACM.

[14] Shengyi Huang and Santiago Ontañón. 2022. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. In *Proceedings of the 35th International FLAIRS Conference*, Vol. 35. doi:10.32473/flairs.v35i.130584

[15] Hadi M. Kamali, Kimia Z. Azar, Farimah Farahmandi, and Mark Tehranipoor. 2023. SheLL: Shrinking eFPGA Fabrics for Logic Locking. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–6.

[16] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. 2023. ChiPFormer: Transferable Chip Placement via Offline Decision Transformer. In *Proceedings of the 40th International Conference on Machine Learning (ICML) (PMLR, Vol. 202)*. 18346–18364.

[17] Yao Lai, Yao Mu, and Ping Luo. 2022. MaskPlace: Fast Chip Placement via Reinforced Visual Representation Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 24019–24030.

[18] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2021. A graph placement methodology for fast chip design. *Nature* 594, 7862 (2021), 207–212. doi:10.1038/s41586-021-03544-w

[19] Kevin E. Murray, Oleg Petelin, Sheng Zhong, Jia Min Wang, Mohamed Eldafrawy, Jean-Philippe Legault, Eugene Sha, Aaron G. Graham, Jean Wu, Matthew J. P. Walker, Hanqing Zeng, Panagiotis Patros, Jason Luu, Kenneth B. Kent, and Vaughn Betz. 2020. VTR 8: High-performance CAD and Customizable FPGA Architecture Modelling. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 13, 2 (2020), 9:1–9:55. doi:10.1145/3388617

[20] Pratyush Nandi, Anubhav Mishra, and Madhav Rao. 2024. GOLDS: Genetic Algorithm-based Optimization of Custom FPGA Architecture Layout Design for Secure Silicon. In *Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI)*. ACM, 92–97. doi:10.1145/3649476.3658743

[21] Chiara Muscari Tomajoli, Luca Collini, Jitendra Bhandari, Abdul Khader Thalakkattu Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2022. ALICE: An Automatic Design Flow for eFPGA Redaction. In *59th ACM/IEEE Design Automation Conference (DAC)*. 781–786. doi:10.1145/3489517.3530543

[22] Nan Wu et al. 2023. GAMORA: Graph Learning based Symbolic Reasoning for Large-Scale Boolean Networks. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.